

CP02 — Prompt Toolkit

FIAP — Prompt Engineering & Artificial Intelligence Checkpoint 02 — Prompt Toolkit

Integrantes

- Luiz Eduardo Correia Garbeloto — RM: 567417
- Eduardo Luiz — RM: RM567662
- Miguel Bezerra — RM: 566763
- Emanuel Nabarrete — RM: 566931
- Lucas Mota — RM: 566670

Atendimento ao cliente e análise de textos corporativos.

Repositório GitHub

<https://github.com/Emanuelnabarrete/Checkpoint-02---PromptToolkit>

2. Contexto

Com o crescimento do uso de Inteligência Artificial em ambientes corporativos, tornou-se essencial encontrar formas eficientes de estruturar prompts para obter respostas mais precisas, consistentes e econômicas.

Empresas utilizam LLMs para tarefas como:

- Classificação de sentimentos;
- Extração de informações;
- Sumarização de textos;
- Geração de respostas automatizadas.

Porém, diferentes técnicas de prompting produzem resultados distintos dependendo da tarefa executada.

Problema

O principal problema enfrentado é descobrir automaticamente qual técnica de prompting apresenta melhor desempenho para cada tipo de tarefa.

Além disso, muitas empresas não possuem ferramentas para:

- Comparar qualidade das respostas;
- Medir consumo de tokens;
- Avaliar consistência;
- Analisar custo computacional;
- Recomendar automaticamente a melhor abordagem.

Solução Desenvolvida

O grupo desenvolveu o **PromptToolkit**, um sistema modular em Python:

- Aplicar automaticamente 4 técnicas de Prompt Engineering;
- Comparar os resultados gerados pelo modelo;
- Medir desempenho e custo;
- Gerar relatórios automáticos;
- Recomendar a técnica mais eficiente para cada tarefa.

Implementamos:

1. Zero-Shot Prompting;
2. Few-Shot Prompting;
3. Chain-of-Thought (CoT);
4. Role Prompting.

O sistema utiliza o modelo **gpt-oss:120b** via **Ollama API**, executado localmente.

3. Estrutura Geral

O projeto foi desenvolvido utilizando arquitetura modular para facilitar manutenção, reutilização de código e escalabilidade.

Fluxo do Sistema

```
inputs.json
  ↓
prompt_builder.py
  ↓
techniques.py
(ZS / FS / CoT / Role)
  ↓
llm_client.py
  ↓
evaluator.py
  ↓
report.py
  ↓
CSV + gráficos + recomendação
```

Organização de Pastas

```
prompt-toolkit/  
├── README.md  
├── requirements.txt  
├── .env.example  
├── main.py  
├── src/  
│   ├── llm_client.py  
│   ├── prompt_builder.py  
│   ├── techniques.py  
│   ├── tasks.py  
│   ├── evaluator.py  
│   └── report.py  
├── data/  
│   ├── inputs.json  
│   └── examples.json  
├── prompts/  
│   ├── system_prompts.json  
│   └── templates.json  
├── output/  
│   ├── resultados.csv  
│   └── graficos/  
└── docs/
```

Responsabilidade dos Módulos

llm_client.py

Responsável pela comunicação com a API do Ollama.

Funções principais:

- Envio de prompts;
- Controle de temperatura;
- Tratamento de erros;
- Medição de tempo de resposta.

prompt_builder.py

Responsável pela construção estruturada dos prompts.

Funções principais:

- Separação entre instrução e dados;
- Inserção de exemplos;
- Adição de Chain-of-Thought.

techniques.py

Implementa as 4 técnicas de prompting.

tasks.py

Define tarefas do domínio e parâmetros necessários.

evaluator.py

Executa medições de:

- Tokens;
- Acurácia;
- Consistência;
- Tempo de resposta.

report.py

Gera:

- CSV comparativo;
 - Gráficos;
 - Recomendação automática.
-

4. Linguagem

- Python 3.10+

Modelo de IA

- gpt-oss:120b
- Execução local via Ollama

Bibliotecas Utilizadas

Biblioteca	Finalidade
requests	Comunicação com API Ollama
pandas	Manipulação de dados
matplotlib	Geração de gráficos
tiktoken	Contagem de tokens
json	Manipulação de arquivos JSON

Biblioteca	Finalidade
time	Medição de desempenho
os	Variáveis de ambiente

Execução do Projeto

Instalação das Dependências

```
pip install -r requirements.txt
```

Executar o Ollama

```
ollama run gpt-oss:120b
```

Rodar o Projeto

```
python main.py
```

5. Tarefas Implementadas

O sistema foi configurado com múltiplas tarefas relacionadas ao domínio corporativo.

Exemplos de tarefas:

- Classificação de sentimento;
- Extração de dados;
- Sumarização;
- Geração de respostas.

Comparações

Técnica	Objetivo
Zero-Shot	Executar sem exemplos
Few-Shot	Melhorar precisão usando exemplos
Chain-of-Thought	Estimular raciocínio passo a passo
Role Prompting	Simular especialistas

Métricas Avaliadas

- Acurácia;
- Quantidade de tokens;
- Tempo de resposta;
- Consistência;
- Custo computacional.

Exemplo de Resultado Comparativo

Técnica	Acurácia	Tokens Médios	Tempo Médio
Zero-Shot	78%	420	1.8s
Few-Shot	89%	610	2.2s
Chain-of-Thought	93%	820	3.1s
Role Prompting	91%	700	2.6s

Análise dos Resultados

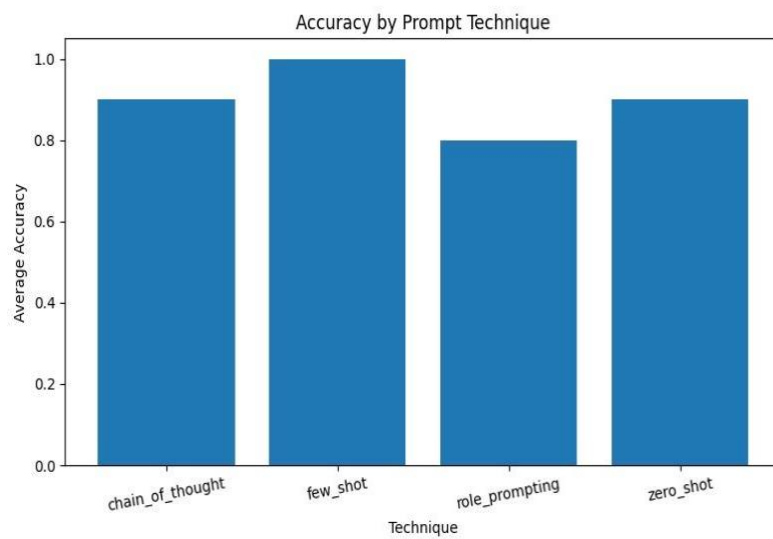
Os testes demonstraram que:

- Few-Shot aumentou significativamente a precisão;
- Chain-of-Thought apresentou melhor capacidade analítica;
- Zero-Shot teve menor custo computacional;
- Role Prompting gerou respostas mais contextualizadas.

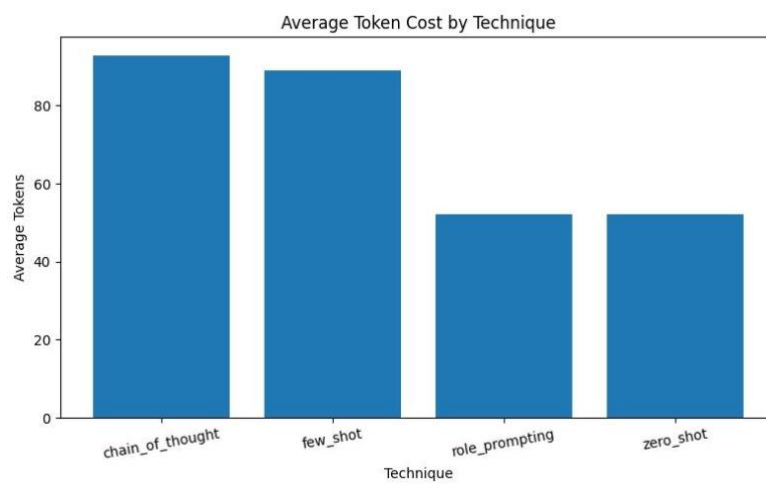
Gráficos Gerados

O sistema gerou automaticamente:

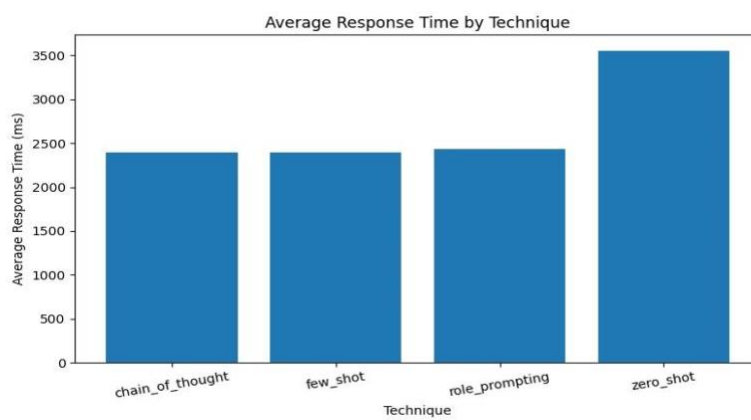
1. Gráfico de acurácia por técnica;



2. Gráfico de custo médio em tokens;



3. Gráfico de consistência por temperatura.



6. Quando Usar Cada Técnica

Técnica	Melhor Uso	Vantagem Principal	Limitação
Zero-Shot	Tarefas simples	Baixo custo	Menor precisão
Few-Shot	Classificação e extração	Alta acurácia	Mais tokens
Chain-of-Thought	Problemas complexos	Melhor raciocínio	Respostas maiores
Role Prompting	Contextos profissionais	Respostas especializadas	Dependência da persona

Recomendações do Grupo

- Utilizar Zero-Shot em tarefas rápidas e simples;
 - Utilizar Few-Shot para classificação de textos;
 - Utilizar Chain-of-Thought em análises complexas;
 - Utilizar Role Prompting quando contexto especializado for necessário.
-

7. Reflexão Final

O desenvolvimento do projeto permitiu compreender na prática como diferentes técnicas de Prompt Engineering impactam diretamente a qualidade das respostas geradas por modelos de linguagem.

Um dos principais aprendizados foi perceber que pequenas mudanças na estrutura do prompt podem alterar significativamente:

- Precisão;
- Clareza;
- Consistência;
- Custo computacional.

O grupo observou que a técnica **Chain-of-Thought** apresentou os melhores resultados em tarefas analíticas, enquanto **Few-Shot** demonstrou excelente equilíbrio entre custo e qualidade.

Além disso, o projeto reforçou a importância de:

- Organização modular;

- Engenharia de prompts;
 - Avaliação quantitativa de respostas;
 - Estruturação de dados.
-

Conclusão

O PromptToolkit demonstrou ser uma ferramenta eficiente para comparar técnicas de Prompt Engineering e auxiliar desenvolvedores na escolha da melhor estratégia para diferentes tarefas.

A aplicação conseguiu integrar análise de desempenho, automação de testes e geração de relatórios em um único sistema modular e reutilizável.

O projeto também permitiu aplicar conceitos importantes de:

- Python modular;
- APIs locais;
- Engenharia de prompts;
- Avaliação de modelos de IA;
- Estruturação de software.